

クイックスタート

本文書では、minitype の基本的な使い方を説明します。

文書を生成する

minitype パッケージから minitype 関数をインポートし、文書の内容とスタイルを指定して呼び出します。戻り値のオブジェクトに対して .save() を呼び出すと PDF または PNG ファイルが出力されます。

```
import { h1, minitype, p } from
"@minitype/minitype";

await minitype([
  {
    body: [h1("Hello World!"),
p("minitype のテストです")],
  },
]).save("hello.pdf");
```

minitype 関数の引数と戻り値は以下の通りです。

```
minitype(
  groups: Group[], // 文書の内容
  style?: Partial<DocumentStyle>, // 文書全体のスタイル
  options?: MiniTypeOptions, // オプション
): {
  save(path: string):
  Promise<string[]> // PDF または PNG
  として保存 (.pdf または .png)
  toPdf(): Promise<Uint8Array>
  // PDF を Uint8Array として返す
  toImages(): Promise<Uint8Array[]>
  // 各ページの PNG を Uint8Array 配列
  として返す
  getLayout():
  Promise<BlockLabel[]> // 組版結
```

果のレイアウト情報を返す

```
getPageCount(): Promise<number>
// 組版結果のページ数を返す
getDiagnostics():
Promise<Diagnostic[]> // 組版中に検
出された診断情報を返す
}
```

options では、以下の設定が行えます。

```
{
  outline: true, // PDF のア
  ウトライン (しおり) を生成する
  metadata: { title: "文書タイトル" },
  // PDF メタデータ
  ppi: 350, // PNG 出力時の
  解像度 (既定値: 350)
  blockTransformers: [], // デ
  フォルトの BlockTransformer に追加で適用
  する配列
  disableDefaultTransformers:
  false, // headingTransformer /
  captionTransformer /
  footnoteTransformer の自動適用を無効に
  する
  fontDir: "fonts", // フォント
  ファイルの格納ディレクトリ (Node.js のみ)
  fonts: [], // ブラウザ向け
  のフォントデータ配列
  browserFiles: [], // ブラウザ
  向けのファイルデータ配列 (figure 等)
}
```

headingTransformer, captionTransformer, footnoteTransformer はデフォルトで自動的に適用されます。数式番号を付与する mathNumberTransformer は自動適用されないため、必要に応じて blockTransformers に追加します。

```
import { mathNumberTransformer,
minitype } from
"@minitype/minitype";

await minitype([ { body: [/* ... */]
}], style, {
  blockTransformers:
[mathNumberTransformer()],
}).save("output.pdf");
```

```
{
  font: "SourceHanSerifJP-Regular";
// SourceHanSerifJP-Regular.otf を使用
}
```

文書スタイルの定義

文書全体のスタイルは minitype 関数の第 2 引数で指定します。

図表、式、ソースコードへの番号付け

図版・表・式・ソースコードは既定で文書全体の通し番号になります。各グループの labelOptions で変更できます。設定は後続のグループへ継承されるため、通常は最初のグループに一度だけ指定します。

```
minitype([
{
  labelOptions: {
    counterResetAt: false, // 文書全体
    で通し番号 (1, 2, ...)
    formatCounter: ({ headingIndex,
index }) =>

`${headingIndex[0]}.${headingIndex[1]}.${index}`,
  },
  body: [/* ... */],
},
{ body: [/* 上の設定を継承 */] },
], style);
```

counterResetAt には見出しレベルの 1 ~ 4, または通し番号にする false を指定します。formatCounter を指定すると、カウンタ値と現在の見出し番号から任意の表示番号を作れます。

フォント

minitype はフォントファイル (.otf, .ttf, .ttc) を読み込んで組版に使用します。options.fontDir で指定したディレクトリ（デフォルトではカレントディレクトリ）にフォントファイルを配置してください。スタイル中でフォントを指定する際は、拡張子を除いたファイル名を使用します。

```
import { cmyk, em, fill, H, minitype,
physical, Q } from
"@minitype/minitype";

await minitype(
[
{
  body: [
    /* ... */
  ],
},
],
{
  size: "A4",
  writingMode: "horizontal",
  padding: physical(20, 25, 20, 25),
  block: {
    paragraph: {
      font:
"SourceHanSerifJP-Regular",
      size: Q(13),
      lineHeight: H(22),
      firstIndent: em(1),
      effects: [fill(cmyk(0, 0, 0,
100))],
    },
    h1: { font:
"SourceHanSerifJP-Bold", size:
Q(18) },
    h2: { font:
"SourceHanSerifJP-Bold", size:
Q(15) },
  },
  gaps: [
```

```
["h1", "paragraph", 8],
["paragraph", "h1", 12],
],
},
).save("output.pdf");
```

DocumentStyle の主な設定項目は以下の通りです。

- size: 用紙サイズ. "A4", "A5", "B5" 等の定型サイズのほか, { width: 210, height: 297 } のように mm 単位で直接指定することもできます。
- writingMode: 書字方向. "horizontal" (横組) または "vertical" (縦組) を指定します。
- padding: 版面の余白. physical(上, 右, 下, 左) で物理方向, logical(ブロック始端, インライン始端, ブロック終端, インライン終端) で論理方向の指定が可能です。値が 2 つの場合は physical(上下, 左右) のように省略できます。
- block: 各ブロック要素の既定スタイル. paragraph, h1 ~ h4, li1 ~ li3, code, footnote 等, ブロックの種類ごとに設定できます。
- command: インラインコマンドの既定スタイル. コマンド名をキーとして, フォントや色等を設定します。
- gaps: ブロック間のアキ. [前のブロック種別, 後のブロック種別, アキ (mm)] の形式で指定します。"fallback" は未指定の組み合わせすべてに適用されます。

単位

minitype では, 以下のヘルパ関数を使用して単位を指定できます。

- Q(n): 級 (1Q = 0.25 mm)。Q(13) は 3.25 mm です。
- H(n): 齒 (1H = 0.25 mm)。実質的に Q と同等の役割を果たします。
- em(n): 文字サイズに対する相対値。em(1) は 1 文字分です。
- cm(n): センチメートル。cm(2) は 20 mm です。
- ratio(n): 親要素に対する割合。ratio(0.8) は親幅の 80% です。
- fr(n): 残りの空間に対する比率 (CSS の fr と同様)。

版面の自動計算

1 行あたりの文字数と行数から余白を自動計算する calculatePhysicalPadding 関数も用意されています。

```
import { calculatePhysicalPadding,
Q, H } from "@minitype/minitype";

const padding =
calculatePhysicalPadding(
  "A5", // 用紙サイズ
  39, // 1 行あたりの文字数
  17, // 行数
  Q(11), // 文字サイズ
  H(18), // 行送り
  "vertical", // 書字方向
);
```

文書定義

minitype が処理する文書の内容は, グループ (Group) の配列として構成されます。各グループはページ設定やスタイルを共有するまとまりであり, body に含まれるブロック要素の列が文書の内容を形成します。

ブロック要素

文書の内容はブロック要素の配列で表現されます。以下のブロック要素が用意されています。

段落

p 関数で段落を作成します。引数には文字列, 文字列の配列, またはインライン要素の二次元配列を渡せます。タグ付きテンプレートリテラルとしても呼び出せます。

```
p("本文のテキストです。");
p("1 行目 \n2 行目"); // 改行を含む文字列
p(["テキスト", bold, "太字"]); // インライン要素を含む場合
p` テンプレートリテラルで ${bold} 書ける `;
```

見出し

h1 から h4 までの見出しを使用できます。

```
h1(" 第 1 章 はじめに ");
h2("1.1 背景 ");
h3("1.1.1 先行研究 ");
```

見出しに番号を付けない場合は { unnumbered: true } を渡します。

```
h2(" 参考文献 ", { unnumbered: true });
```

リスト

li1, li2, li3 で順序なしリスト, ol1, ol2, ol3 で順序付きリストを作成できます。それぞれ 3 段階のレベルに対応しています。

```
li1(" 最初の項目 ");
li2(" 入れ子の項目 ");
ol1(" 番号付きの項目 ");
ol2(" 番号付きの入れ子の項目 ");
```

ol1 ~ ol3 は既定のマーカースタイルによって番号が付与されます。リストのマーカーは style.marker でカスタマイズできます。

コードブロック

code 関数でコードブロックを作成します。lang 引数に言語名を渡すとシンタックスハイライトが適用されます。label を付与すると、autoref で「ソースコード 1」のように参照できます。

```
code("const x = 42;", "javascript");
{ ...code(["fn main() {", '
println!("Hello");', "}], "rust"),
  label: "code:hello" };
```

脚注

脚注は、本文中の参照 (fn) と脚注本体 (footnote) の組み合わせで実現します。

```
import { fn, footnote, minitype, p }
from "@minitype/minitype";

await minitype([
```

```
{
  body: [
    p([" 本文のテキスト ", fn("note1", {
      before: "*" }), " です. "]]),
    footnote("note1", " この脚注はページ
下部に表示されます. "),
  ],
},
]).save("output.pdf");
```

fn は InlineExtender で、本文中に脚注番号を挿入します。脚注本体の先頭への対応する番号の付与は footnoteTransformer が担い、デフォルトで自動的に適用されます。

数式

math 関数で LaTeX 記法の数式ブロックを作成します。

```
math(["E = mc ^ 2"]);
math(["\\int_0^\\infty e ^ {-x} dx =
1"]);
```

インライン数式には inlineMath ヘルパ関数を使用します。

```
p([
  [
    " アインシュタインの式 ",
    inlineMath("E=mc ^ 2"),
    " は有名です. ",
  ],
]);
```

セクション

section は透過的なコンテンツブロックです。レイアウト (gap 等) に影響を与えず、子ブロックのスタイルを一括で上書きするために使用します。

```
import { h2, p, Q, section } from
"@minitype/minitype";

section(
  [h2(" 見出し "), p(" 本文 ")],
  { paragraph: { size: Q(11) } }, // 子
  ブロックのスタイルを上書き
```

```
);
```

カウンタのリセット

resetLabel で図表番号等のカウンタをリセットします。

```
import { resetLabel } from
"@minitype/minitype";

resetLabel(); // 全カウン
タをリセット
resetLabel(["image", "table"]);
// 画像・表のカウンタのみリセット
resetLabel(["h1"]); // h1
以下の見出しカウンタをリセット
```

インライン要素

ブロック要素の中身はインライン要素の配列で表現されます。インライン要素には以下の種類があります。

- 文字列：もっとも基本的な要素です。
- コマンド (Command)：スタイルの変更やリンク等、テキストに付加情報を与えます。

```
// 太字のコマンド
const bold: Command = {
  type: "command",
  name: "bold",
  body: ["太字のテキスト"],
};
```

コマンドの name に対応するスタイルを DocumentStyle.command で定義しておくと、そのスタイルが自動的に適用されます。

```
await minitype([
  { body: [p(["通常テキスト", bold, "通常テキスト"])] },
  { command: {
    bold: { font:
      "SourceHanSerifJP-Bold" },
  } },
]).save("output.pdf");
```

インラインヘルパ関数を使用すると、コマンドを簡単に作

成できます。

```
import {
  cmyk,
  color,
  command,
  del,
  em,
  fontSize,
  scale,
  u,
  url,
} from "@minitype/minitype";

command("テキスト", { name: "bold" });
// コマンド
color("赤いテキスト", cmyk(0, 100,
100, 0)); // 文字色
fontSize("大きいテキスト", 10); // 文字
サイズ (mm)
scale("小さいテキスト", em(0.8)); //
相対スケール
del("取り消し線"); // 打ち消し線
u("下線"); // 下線
url("リンクテキスト",
"https://example.com"); // URL リンク
```

- 強制改行 (ForceBreak)：fbr() で改行を挿入します。両端揃えが指定されている場合、直前の行は両端揃えとなります。
- カーニング (Kerning)：kern(em) で文字間隔を em 単位で調整します。
- 行分割禁止 (NoBreak)：noBreak() で挿入位置の前後での行分割を禁止します。
- 分離禁止 (NoSplit)：noSplit() で挿入位置の前後での行分割およびトラッキング挿入を禁止します。
- インライングラフィック (InlineGraphic)：inlineGraphic(src, options?) でインライン画像を挿入します。

```
import { inlineGraphic, em } from
"@minitype/minitype";

p(["テキスト中に ",
inlineGraphic("icon.png", { size:
```

```
em(1) }}, " 画像を挿入 "]]);
```

- インライン数式 (InlineMath) : `inlineMath(latex, size?)` で LaTeX のインライン数式を挿入します。
- 水平ボックス (Hbox) : `hbox(width, body)` で指定した幅のボックスを作成します。リーダー (点線等) にも使用できます。

```
import { hbox, fr } from
"@minitype/minitype";

hbox(fr(1), " ..... "); // 余白をリーダーで埋める
hbox(30, [" 左寄せコンテンツ "]); // 30 mm のボックス
```

- ルビ (Ruby) : `ruby(base, ruby)` でルビを付与します。

```
import { ruby } from
"@minitype/minitype";

p([[ruby(" 薔薇 ", " ばら "), " が咲いた "]]);
p([...ruby(" 薔 ", " ば ", " 薇 ", " ら ")]]); // 文字ごとにルビを付ける
```

InlineExtender

InlineExtender は、組版処理時にインライン要素に展開される関数です。相互参照やページ番号の挿入に使用します。

- `ref(label)` : ラベルの参照番号を挿入します。リンク付きです。
- `autoref(label)` : 「図 1」「ソースコード 2」のように、種類と番号を挿入します。
- `pageref(label)` : ラベルが存在するページ番号を挿入します。
- `page` : 現在のページ番号を挿入します。
- `totalPages` : 総ページ数を挿入します。
- `fn(label, options?)` : 脚注番号を挿入します。

- `marker(before, after)` : リストのマーカー (番号等) を挿入します。
- `headingInPage(options?)` : フロー内で使用し、現在ページに出現した見出しのテキストを挿入します (柱として使用)。

`url(href, body?)` はハイパーリンク付きの Command を返すヘルパ関数です (InlineExtender ではありません)。

タグ付きテンプレートリテラル

`p`, `h1` ~ `h4`, `li1` ~ `li3`, `ol1` ~ `ol3` 等のブロック生成関数はタグ付きテンプレートリテラルとしても呼び出せます。テンプレートにインライン要素を直接埋め込みます。

```
import { command, il, kern, p } from
"@minitype/minitype";

const emphasis = command(" 重要な語句 ", { name: "bold" });

p`本文中に ${emphasis} や
${kern(-0.1)} カーニングを埋め込める。`;
```

インライン要素の配列を作成するには `il` タグを使用します。

```
const inline = il`テキスト ${emphasis}
テキスト`;
```

改ページ・改段

`newpage()` でページを改め、`clearpage()` で保留中のフロートをすべて出力してから改ページします。`newcolumn()` で段を改めます。`newcolumn()` は 1 段組の場合は改ページとして動作します。

```
import { newpage, clearpage,
newcolumn, p } from
"@minitype/minitype";

[p("1 ページ目"), newpage(), p("2 ページ目"), clearpage(), p("次のページ")];
```

`vspace(mm)` でブロック方向に任意の空白を挿入することもできます。

図版

`image` 関数で画像ファイルを挿入します。PNG, JPG, SVG, PDF に対応しています。 `figure` ヘルパ関数を使用すると、画像とキャプションをまとめて配置できます。

```
import { image, ratio } from
"@minitype/minitype";

image("image.png", { width:
ratio(0.8), align: "center" });
```

`width` には mm 単位の数値、または `ratio()` で親要素に対する割合を指定できます。テキストの回り込みには `box` の `wrap` プロパティを使用します。

キャプション

画像や表にキャプションを付与するには `caption` 関数を使用します。 `captionTransformer` はデフォルトで自動的に適用されるため、自動採番が有効になっています。

```
import { caption, image, minitype,
ratio } from "@minitype/minitype";

await minitype([
{
  body: [
    image("chart.png", { width:
ratio(0.6) }),
    caption(" 実験結果のグラフ "),
  ],
},
]).save("output.pdf");
```

表

`table` 関数で表を作成します。内容は [行] [列] の二次元配列で、各セルは `cell` 関数にブロック要素を渡して作成します。

```
import { cell, cmyk, fr, p, ratio,
rgb, table } from
"@minitype/minitype";

table(
[
  [cell(p(" 名前 ")), cell(p(" 年齢
")), cell(p(" 所属 "))],
  [cell(p(" 田中 ")), cell(p("30")),
cell(p(" 開発部 "))],
  [cell(p(" 佐藤 ")), cell(p("25")),
cell(p(" 企画部 "))],
],
{
  columnWidths: [fr(1), 20,
ratio(0.3)],
  horizontalBorders: (rowIndex,
rowCount) => {
    if (rowIndex === 0 || rowIndex ===
rowCount) {
      return { color: cmyk(0, 0, 0,
100), styles: [{ width: 1.5 }] };
    }
    return { color: cmyk(0, 0, 0,
100), styles: [{ width: 0.5 }] };
  },
  background: (rowIndex) =>
    rowIndex === 0 ? cmyk(0, 0, 0, 10)
: rgb(255, 255, 255),
},
);
```

`columnWidths` にはセルごとの幅を指定します。mm 単位の数値、`ratio()` による割合、`fr()` による比率を混在させることができます。 `horizontalBorders` と `verticalBorders` には罫線のスタイルを、`background` にはセルの背景色を、それぞれ値または関数で指定します。

図形

`rect` で矩形、`ellipse` で楕円を描画します。幅と高さは mm 単位です。

```
import { cmyk, ellipse, fill,
physical, rect, solid } from
"@minitype/minitype";
```

```
rect(60, 40, {
  background: [fill(cmyk(0, 0, 0,
20))],
  align: "center",
});
ellipse(50, 50, {
  border: physical(solid(0.5,
cmyk(0, 0, 0, 100))),
});
```

ボックスを描画する

ボックス (Box) は、子ブロック要素をまとめるコンテナ要素です。段組やフロート配置を実現するために使用します。

段組

ボックスの `style.columns` に段数を指定すると、内容が多段組でレイアウトされます。

```
import { box, p } from
"@minitype/minitype";

box([p("1 段目の内容 ..."), p("2 段目に
流れ込む内容 ...")], {
  columns: 2,
  columnGap: 5,
  columnFill: "balance",
});
```

`columnGap` は段間の幅 (mm)、`columnBorder` は段間に引く罫線のスタイルです。`columnFill: "balance"` を指定すると、ボックスがそのページ内で完結する場合に各段の高さを揃えます。既定値は `columnFill: "auto"` で、ページ下部まで順に流れ込みます。ボックスは既定で途中改ページを避け、現在の段に収まらない場合は次の段に送られます。途中改ページを許可するには `splitable: true` を指定します。

フレックスボックス

`flexbox` 関数は、複数のボックスをインライン方向（横書きなら横方向）に並べて配置します。

```
import { box, flexbox, p, ratio } from
"@minitype/minitype";

flexbox([
  box([p(" 左カラム ")], { inlineSize:
ratio(0.3) }),
  box([p(" 右カラム ")],
]);
```

各ボックスの `inlineSize` で幅を指定します。`inlineSize` が省略されたボックスは、残りの幅を均等に分割します。`inlineSize` には mm 単位の数値、`ratio()` による割合、`fr()` による比率を使用できます。

フレックスボックスはページをまたぐことができません。

フロート

`float("top", blocks)` または `float("bottom", blocks)` を使用すると、ブロックをページの上端または下端に浮動配置します。

```
import { caption, float, image } from
"@minitype/minitype";

float("top", [image("chart.png"),
caption(" 実験結果 ")]);
```

現在のページに配置できない場合は、次のページに持ち越されます。

装飾

ボックスには `background` (背景エフェクト)、`border` (罫線)、`borderRadius` (角丸)、`padding` (内側の余白)、`margin` (外側の余白) を設定できます。

```
import { box, cmyk, fill, logical, p,
physical, solid } from
"@minitype/minitype";

box([p(" 囲み記事の内容 ")], {
  padding: physical(4, 6, 4, 6),
  background: [fill(cmyk(0, 0, 0,
5))],
```

```
border: logical({
  blockStart: solid(0.5, cmyk(0, 0,
0, 80)),
  blockEnd: solid(0.5, cmyk(0, 0, 0,
80)),
}),
borderRadius: 2,
});
```

罫線は `solid( 幅, 色 )`, `dashed( 幅, 色 )`, `dotted( 幅, 色 )`, `double( 幅, 色 )` 等のヘルパ関数で作成できます。

柱・ノンブルを挿入する

ヘッダ（柱）やフッタ（ノンブル）は、フロー（Flow）として定義します。フローは本文とは独立してページ上に配置されるブロック要素群です。

```
import { minitype, p, page } from
"@minitype/minitype";

await minitype([
{
  body: [
    p("本文のテキスト..."),

    // 柱（ヘッダ）
    {
      type: "flow",
      position: "pillar",
      blockOffset: -3,
      blocks: [p("文書タイトル", {
firstIndent: 0 })],
    },

    // ノンブル（ページ番号）
    {
      type: "flow",
      position: "nombre",
      blockOffset: 0,
      blocks: [p([[page]], { align:
"center", firstIndent: 0 })],
    },
  ],
},
],
},
```

```
]).save("output.pdf");
```

`position` には 3 種類の基準位置があります。

- "pillar" : 版面上端（縦書きでは右端）を基準とする相対位置です。
- "nombre" : 版面下端（縦書きでは左端）を基準とする相対位置です。
- "page" : ページ左上を原点とする絶対位置です。

`blockOffset` と `inlineOffset` でフローの位置を微調整できます。 `inlineSize` でフローの幅を指定することも可能です。

`page` プロパティを使用すると、特定のページにのみフローを表示できます。ページ番号を直接指定するか、関数で条件を記述します。

```
{
  type: "flow",
  position: "nombre",
  page: (pageIndex) => pageIndex >= 1,
  // 2 ページ目以降にのみ表示
  blocks: [p([[page]], { align:
"center", firstIndent: 0 })],
}
```

柱に現在ページの見出しを表示するには `headingInPage InlineExtender` を使用します。

```
import { headingInPage } from
"@minitype/minitype";

{
  type: "flow",
  position: "pillar",
  blockOffset: -3,
  blocks: [p([[headingInPage({
level: 2 })]], { firstIndent: 0 })],
}
```

タイトルを挿入する

タイトルページは、グループの仕組みを使用して作成しま

す。タイトル用のグループを本文とは別に定義し、フローで自由な位置に要素を配置します。

```
await minitype([
  // タイトルページ (グループ 1)
  {
    body: [
      {
        type: "flow",
        position: "page",
        inlineOffset: 50,
        blockOffset: 80,
        blocks: [h1(" 文書のタイトル ")],
      },
      {
        type: "flow",
        position: "page",
        inlineOffset: 50,
        blockOffset: 120,
        blocks: [p(" 著者名 ", { align:
"left", firstIndent: 0 })],
      },
    ],
  },
  // 本文 (グループ 2)
  {
    body: [h1(" はじめに "), p(" 本文の内容
...")],
  },
]).save("output.pdf");
```

各グループは独立したページ番号を持つことができ、pageIndex プロパティで開始ページを制御できます。

固定の画像を挿入する

フローの position: "page" を使用すると、ページ上の任意の座標に画像やブロック要素を固定配置できます。背景画像や透かし、装飾等に使用できます。

```
{
  type: "flow",
  position: "page",
  inlineOffset: 0,
  blockOffset: 0,
```

```
  zIndex: -1, // 本文より背面に配置
  blocks: [image("background.png", {
    width: 210 })],
}
```

zIndex を指定すると、要素の前後関係（重ね順序）を制御できます。負の値を指定すると本文の背面に配置されます。